

Unit IV Activity II - 19.08.2026

Q1. Daily Sales Time Series

Program:

```
set.seed(123)

Date <- seq(as.Date("2024-01-01"), as.Date("2025-12-31"), by = "day")

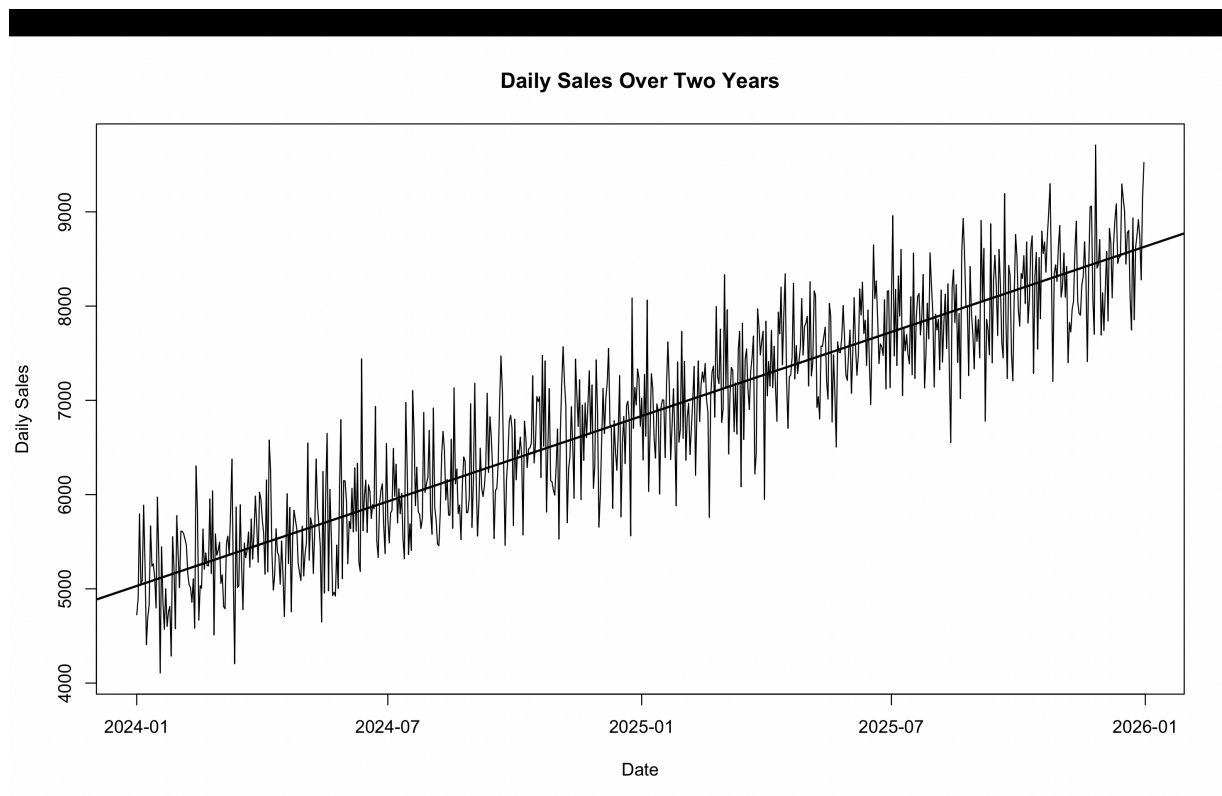
Sales <- 5000 + 5*(1:length(Date)) + rnorm(length(Date), 0, 500)

data <- data.frame(Date, Sales)

plot(data$Date, data$Sales,
      type = "l",
      main = "Daily Sales Over Two Years",
      xlab = "Date",
      ylab = "Daily Sales")

abline(lm(Sales ~ as.numeric(Date), data = data), lwd = 2)
```

Output:



Q2. Server Temperature Time Series

Program:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(10)

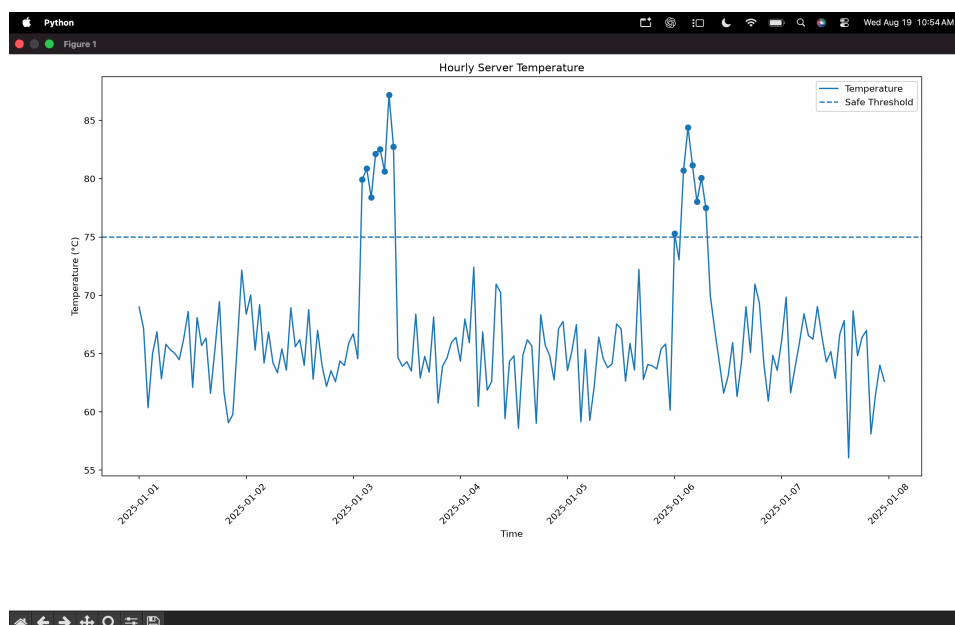
time = pd.date_range("2025-01-01", periods=168, freq="h")
temperature = 65 + np.random.normal(0, 3, 168)
temperature[50:58] += 15
temperature[120:128] += 12

threshold = 75

plt.plot(time, temperature, label="Temperature")
plt.axhline(threshold, linestyle="--", label="Safe Threshold")
plt.scatter(time[temperature > threshold],
            temperature[temperature > threshold])

plt.xlabel("Time")
plt.ylabel("Temperature (°C)")
plt.title("Hourly Server Temperature")
plt.legend()
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Output:



Q3. Tech Stock Price Comparison

Program:

```
set.seed(123)

Date <- seq(as.Date("2025-01-01"), by = "day", length.out = 100)

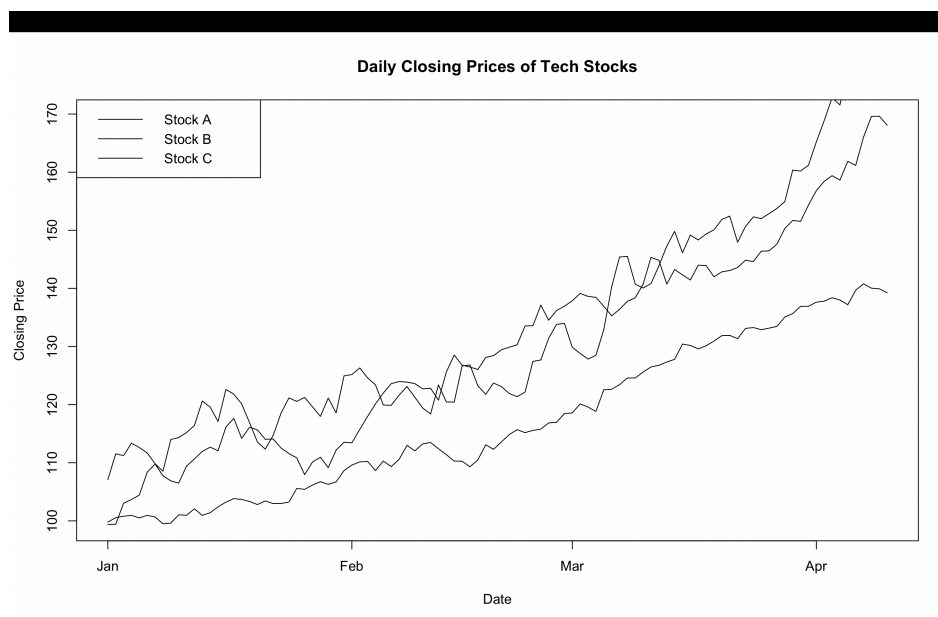
Stock_A <- cumsum(rnorm(100, 0.5, 2)) + 100
Stock_B <- cumsum(rnorm(100, 0.5, 1)) + 100
Stock_C <- cumsum(rnorm(100, 0.5, 3)) + 100

plot(Date, Stock_A,
     type = "l",
     xlab = "Date",
     ylab = "Closing Price",
     main = "Daily Closing Prices of Tech Stocks")

lines(Date, Stock_B)
lines(Date, Stock_C)

legend("topleft",
     legend = c("Stock A", "Stock B", "Stock C"),
     lty = 1)
```

Output:



Comment: Stock B appears least volatile because its price series shows the smallest fluctuations.

Q4. Neighborhood Electricity Consumption

Program:

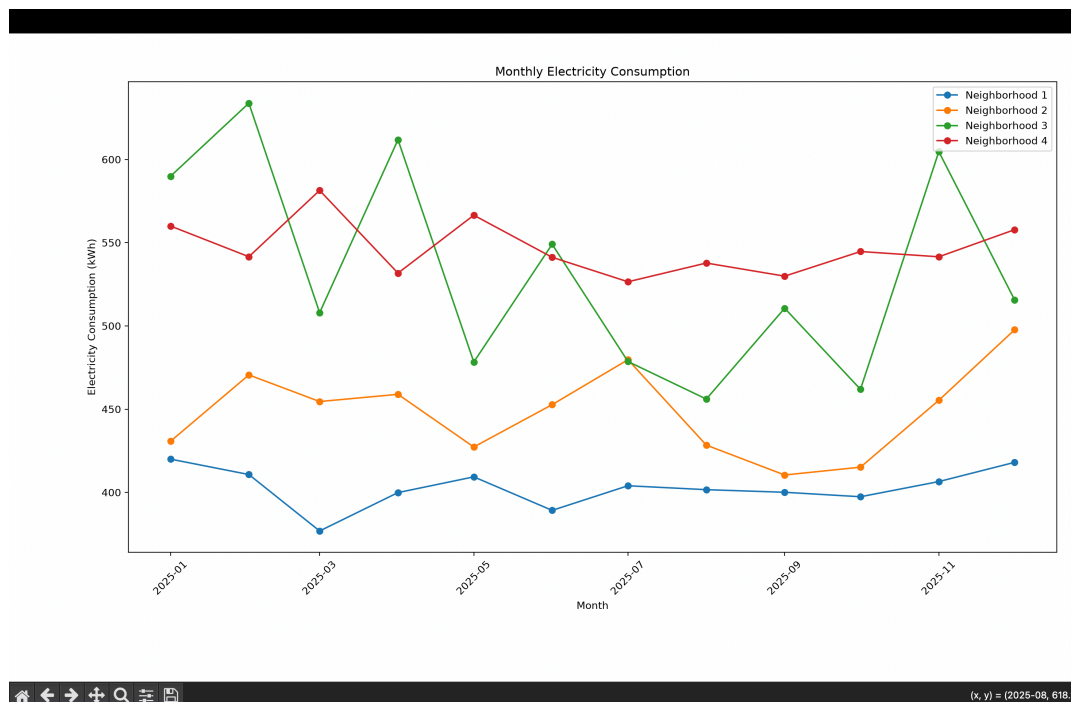
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
np.random.seed(10)
months = pd.date_range("2025-01-01", periods=12, freq="MS")
```

```
N1 = 400 + np.random.normal(0, 15, 12)
N2 = 450 + np.random.normal(0, 20, 12)
N3 = 500 + np.random.normal(0, 80, 12)
N4 = 550 + np.random.normal(0, 25, 12)
```

```
plt.plot(months, N1, marker="o", label="Neighborhood 1")
plt.plot(months, N2, marker="o", label="Neighborhood 2")
plt.plot(months, N3, marker="o", label="Neighborhood 3")
plt.plot(months, N4, marker="o", label="Neighborhood 4")
```

```
plt.xlabel("Month")
plt.ylabel("Electricity Consumption (kWh)")
plt.title("Monthly Electricity Consumption")
plt.legend()
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Output:



Comment: Neighborhood 3 appears to have the most erratic consumption pattern because its values fluctuate more widely.

Q5. Dose-Response Curve

Program:

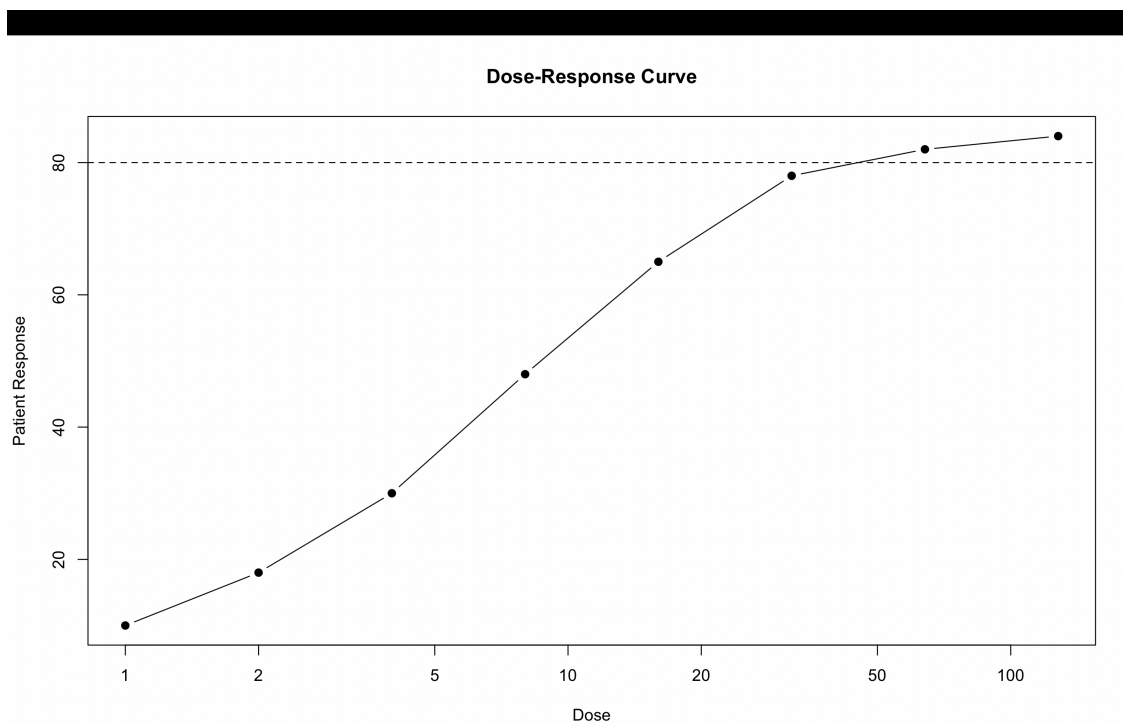
```
set.seed(123)

Dose <- c(1, 2, 4, 8, 16, 32, 64, 128)
Response <- c(10, 18, 30, 48, 65, 78, 82, 84)

plot(Dose, Response,
     type = "b",
     pch = 19,
     log = "x",
     main = "Dose-Response Curve",
     xlab = "Dose",
     ylab = "Patient Response")

abline(h = 80, lty = 2)
```

Output:



The response begins to plateau around a dose of 32–64 units, where further increases in dose produce only small increases in response.

Q6. Fertilizer Dose–Response Curve

Program:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

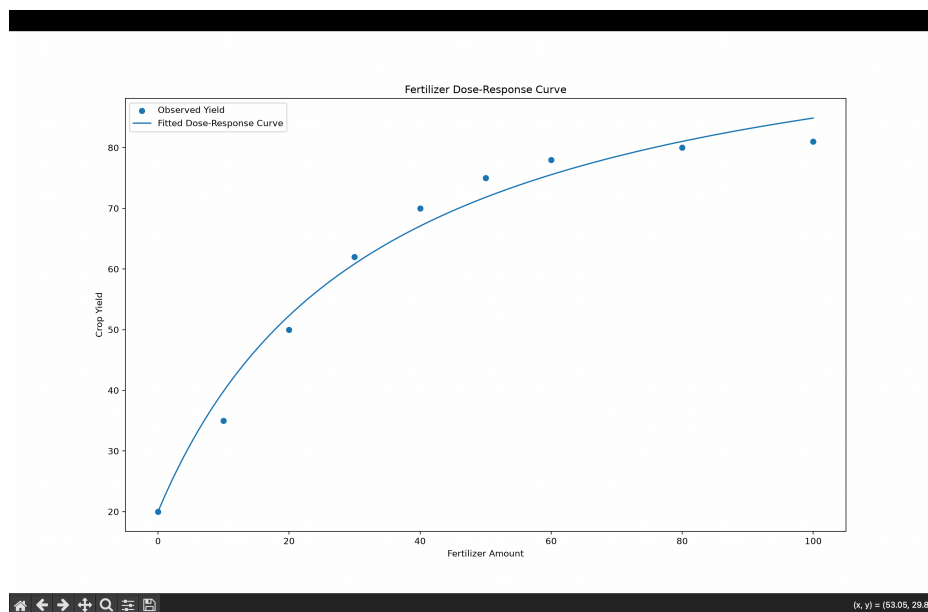
dose = np.array([0, 10, 20, 30, 40, 50, 60, 80, 100])
yield_data = np.array([20, 35, 50, 62, 70, 75, 78, 80, 81])

def response(x, a, b):
    return a * x / (b + x) + 20
params, _ = curve_fit(response, dose, yield_data, p0=[100, 30])
x = np.linspace(0, 100, 200)
y = response(x, *params)

plt.scatter(dose, yield_data, label="Observed Yield")
plt.plot(x, y, label="Fitted Dose-Response Curve")

plt.xlabel("Fertilizer Amount")
plt.ylabel("Crop Yield")
plt.title("Fertilizer Dose-Response Curve")
plt.legend()
plt.show()
```

Output:



The curve begins to level off around 60–80 units, so this range can be considered the optimal dosage range for avoiding unnecessary over-application.

Q7. STL Decomposition

Program:

```
set.seed(123)

Month <- seq(as.Date("2023-01-01"), by = "month", length.out = 36)

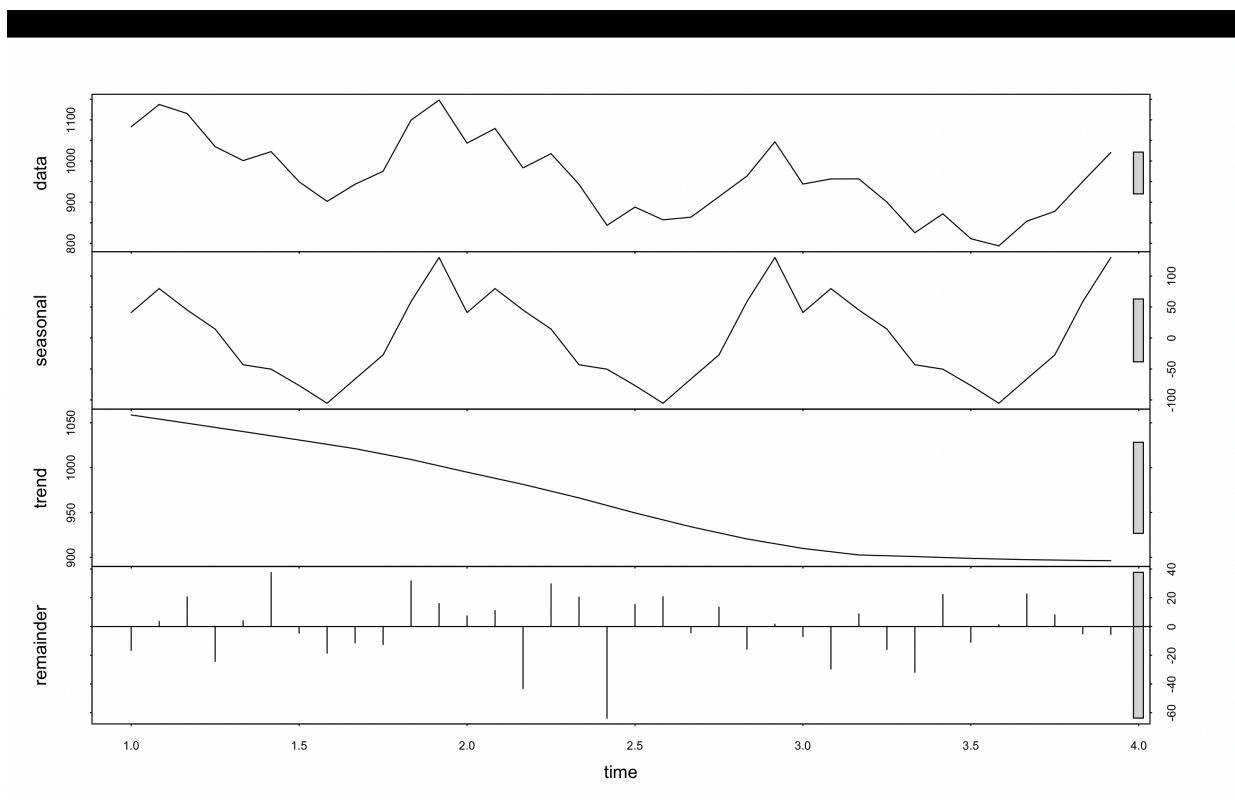
Trend <- seq(1000, 800, length.out = 36)
Seasonal <- rep(c(100, 150, 80, 50, 20, 0, -30, -20, 10, 40, 120, 200), 3)
Sales <- Trend + Seasonal + rnorm(36, 0, 30)

sales_ts <- ts(Sales, frequency = 12)

decomp <- stl(sales_ts, s.window = "periodic")

plot(decomp)
```

Output:



Q8. STL Decomposition of Weekly Users

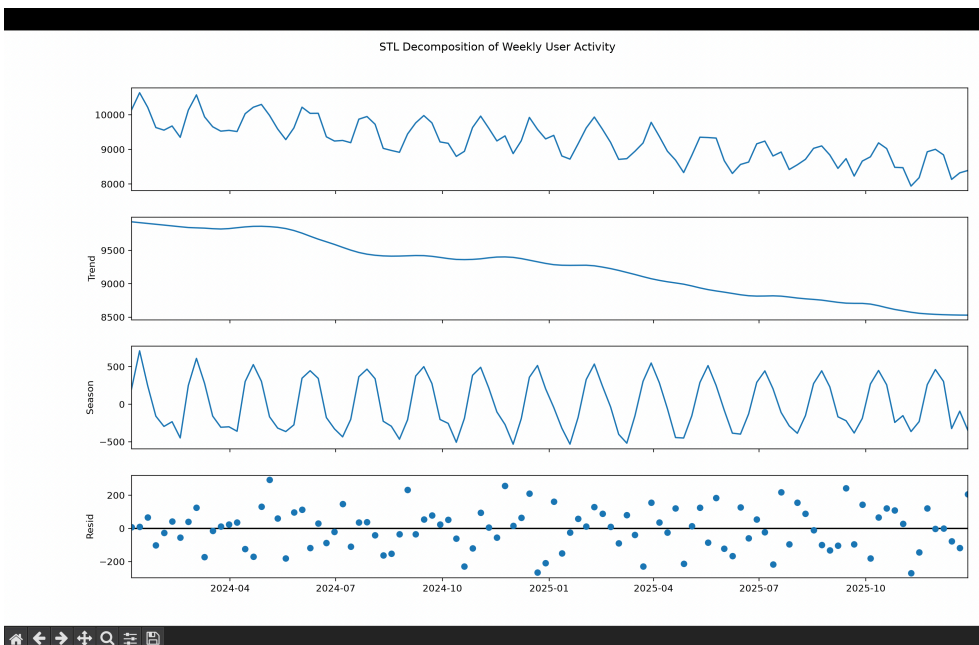
Program:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import STL

np.random.seed(123)

weeks = pd.date_range("2024-01-07", periods=104, freq="W")
trend = np.linspace(10000, 8500, 104)
seasonal_pattern = np.tile(
    [300, 500, 200, -100, -300, -500, -200], 15
)[:104]
users = trend + seasonal_pattern + np.random.normal(0, 150, 104)
data = pd.Series(users, index=weeks)
result = STL(data, period=7).fit()
result.plot()
plt.suptitle("STL Decomposition of Weekly User Activity")
plt.tight_layout()
plt.show()
```

Output:



Interpretation:

The trend component shows a gradual decline in weekly active users from about 10,000 to 8,500.

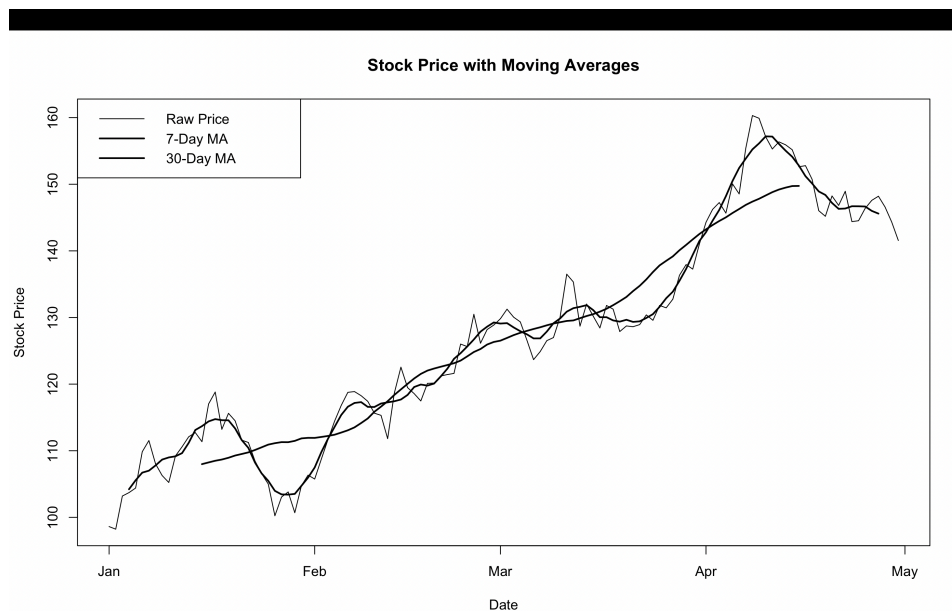
Therefore, the recent dips are not only due to normal seasonal viewing habits. The underlying trend also indicates a real decline in user activity.

Q9. 7-Day and 30-Day Moving Average

Program:

```
set.seed(123)
Date <- seq(as.Date("2025-01-01"), by = "day", length.out = 120)
Price <- 100 + cumsum(rnorm(120, 0.3, 3))
MA7 <- stats::filter(Price, rep(1/7, 7), sides = 2)
MA30 <- stats::filter(Price, rep(1/30, 30), sides = 2)
plot(Date, Price,
     type = "l",
     main = "Stock Price with Moving Averages",
     xlab = "Date",
     ylab = "Stock Price")
lines(Date, MA7, lwd = 2)
lines(Date, MA30, lwd = 2)
legend("topleft",
     legend = c("Raw Price", "7-Day MA", "30-Day MA"),
     lty = 1,
     lwd = c(1, 2, 2))
```

Output:



Explanation:

- 7-day moving average reacts quickly to short-term price changes.
- 30-day moving average is smoother and shows the longer-term trend.
- When the 7-day MA crosses above the 30-day MA, it suggests upward/bullish momentum.
- When the 7-day MA crosses below the 30-day MA, it suggests downward/bearish momentum.

Q10. GDP Detrending

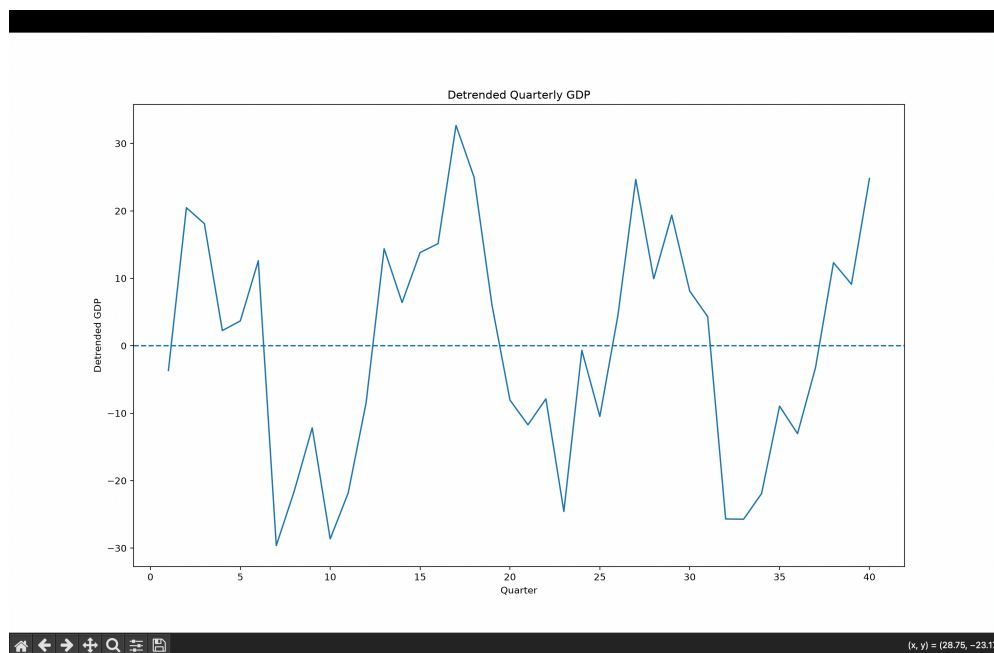
Program:

```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(123)
quarters = np.arange(1, 41)
GDP = 500 + 8*quarters + 20*np.sin(quarters/2) + np.random.normal(0, 8, 40)
coeff = np.polyfit(quarters, GDP, 1)
trend = np.polyval(coeff, quarters)
```

```
detrended = GDP - trend
```

```
plt.plot(quarters, detrended)
plt.axhline(0, linestyle="--")
plt.xlabel("Quarter")
plt.ylabel("Detrended GDP")
plt.title("Detrended Quarterly GDP")
plt.show()
```

Output:



Interpretation

The long-term GDP growth trend has been removed. The remaining fluctuations show short-term business-cycle movements:

- Values above 0 → GDP is above its long-term trend.
- Values below 0 → GDP is below its long-term trend.
- Larger fluctuations indicate stronger short-term changes in economic activity.